

When Correct Code Fails: Evaluating AI-Generated Gameplay Logic in Unity Runtime Systems

Yu-Chi Chuang

“After reading this piece, a practitioner will understand how AI-generated gameplay control logic in Unity works well enough to decide where and how to integrate it into the runtime system without making the mistake of placing logic in the wrong execution loop or relying on brittle equality checks that break gameplay behavior.”

The Scenario

This project is based on a roll-a-ball style Unity prototype in which the player controls a sphere to collect colored pickups and interact with walls based on color matching. The intended gameplay rule is simple: when the player collects a colored pickup, the sphere changes color, and the player can pass through walls that match that color.

To accelerate development, an AI system was used to generate the player controller script, including movement logic, collision handling, and color comparison. The assumption behind this approach is that AI-generated code can rapidly produce functional gameplay systems without requiring extensive manual implementation. However, the key question explored in this project is not whether AI can generate working code, but whether that code can be directly integrated into a real-time physics-based game system without introducing unintended behavior. The scenario focuses specifically on how AI-generated gameplay logic interacts with Unity’s runtime execution model and whether it preserves the intended player experience.

The Mechanism

The AI application operates by transforming natural language prompts into executable C# scripts for Unity. The generated script includes several components: force-based movement using a Rigidbody, input handling through Unity’s axis system, collision detection for pickups and walls, and color comparison logic for gameplay rules.

From an input-output perspective, the developer provides a description such as “move a ball using physics and allow it to pass through walls of the same color,” and the AI produces a script that appears structurally correct. The script compiles successfully and executes without syntax errors, giving the impression of a valid implementation.

However, Unity’s runtime behavior depends not only on code correctness but also on how and where that code is executed. Physics-based movement must be applied within `FixedUpdate` to align with Unity’s physics engine. Similarly, color comparisons must account for floating-point imprecision and material differences. The AI-generated script does not inherently understand these engine-specific constraints. As a result, the correctness of the generated code is superficial—it satisfies syntax and structure but may violate runtime expectations.

The Design Decision

The central design decision in this project is not whether to use AI-generated code, but how to integrate it into the game engine's execution model. Specifically, two decisions prove critical:

1. Whether to apply physics forces in Update or FixedUpdate
2. Whether to compare colors using exact equality or tolerance-based comparison

These decisions are non-trivial because they directly affect runtime behavior. Applying force in Update ties physics to frame rate, while FixedUpdate ensures consistent physics simulation. Similarly, exact color equality assumes numerical identity, while tolerance-based comparison aligns better with how colors are perceived in the game. The AI-generated version chose to apply force in Update and compare colors using direct equality. Both choices appear reasonable at a surface level, but they reflect a lack of alignment with Unity's execution model. The developer must intervene and reinterpret the AI output, treating it as a draft rather than a final solution.

The Failure Case

Two primary failures were observed in the AI-generated implementation.

First, movement forces were applied within the Update loop instead of FixedUpdate. Because Update runs once per frame, the amount of force applied per second becomes dependent on frame rate. This results in inconsistent and often excessive acceleration, making the ball difficult to control. The behavior varies across hardware and frame conditions, leading to unstable gameplay.

Second, color matching was implemented using exact equality (`a == b`). In practice, colors that appear visually identical may not have identical numerical values due to material instances, lighting conditions, or floating-point precision. As a result, the player can collect a pickup, match the color of a wall visually, and still fail to pass through it. This breaks the core gameplay rule and creates a mismatch between player expectation and system behavior.

These failures demonstrate that AI-generated code can be syntactically correct yet semantically incompatible with the runtime system. The issue is not the absence of functionality, but the incorrect placement and interpretation of that functionality within the engine.

The Exercise

To reproduce the failure, two modifications can be made.

First, move the physics-based movement logic from FixedUpdate to Update. Then increase the movement speed or reduce drag in the Rigidbody settings. The ball will begin to accelerate uncontrollably, demonstrating the instability caused by frame-dependent force application.

Second, replace the tolerance-based color comparison with direct equality (`return a == b`). After collecting a pickup, attempt to pass through a wall of the same visible color. The player will observe that the wall does not respond as expected, revealing the brittleness of exact comparison.

These exercises highlight how small implementation choices can lead to significant runtime failures. They reinforce the central claim of this project: AI is not a magic layer, but a pipeline decision. Where the AI-generated logic is placed, and how it is constrained, determines whether the system behaves correctly or fails.

Conclusion

This project demonstrates that AI-generated gameplay code must be treated as a starting point rather than a final solution. While AI can produce code that compiles and appears functionally complete, it lacks an understanding of engine-specific execution models and player-facing behavior.

The failures observed—frame-dependent physics instability and brittle color comparison—illustrate that correctness in code does not guarantee correctness in gameplay. The responsibility remains with the developer to interpret, constrain, and integrate AI-generated output appropriately.

Ultimately, the value of AI in game development lies not in replacing design decisions, but in accelerating iteration. The developer must still decide where the logic belongs and how it should behave within the system. This project serves as a concrete example of the course's central claim: AI is a pipeline decision, not a magic layer.